

UNIVERSIDAD INDUSTRIAL DE SANTANADER
FACULTAD DE INGENIERÍAS FISICOMECAÑICAS
ESCUELA DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

PLAN DE TRABAJO DE GRADO

FECHA DE PRESENTACIÓN: Bucaramanga, 06 de Abril de 2026

TITULO: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning

MODALIDAD: Trabajo de Investigación

AUTOR: Yeison Adrian Caceres Torres - 2220075

AUTOR: Ricardo Andres Perez Porras - 2220078

DIRECTOR: Gilberto Javier Diaz Toro, Ph.D.

ENTIDAD INTERESADA: Escuela de Ingeniería de Sistemas e Informática

TABLA DE CONTENIDO

<u>1</u>	<u>Introducción</u>	2
<u>2</u>	<u>Planteamiento Y Justificación Del Problema</u>	4
<u>3</u>	<u>Objetivos</u>	5
<u>3.1</u>	<u>Objetivo General</u>	6
<u>3.2</u>	<u>Objetivos Específicos</u>	6
<u>4</u>	<u>Marco De Referencia</u>	6
<u>4.1</u>	<u>Marco Conceptual</u>	7
<u>4.1.1</u>	<u>Termodinámica del Silicio y Potencia CMOS</u>	7
<u>4.1.2</u>	<u>Modelo Roofline y regímenes de limitación del rendimiento</u>	7
<u>4.1.3</u>	<u>Telemetría microarquitectónica</u>	8
<u>4.1.3.1</u>	<u>Unidades de Monitorización de Rendimiento (PMU) y Contadores de Rendimiento por Hardware</u>	9
<u>4.1.3.2</u>	<u>Métricas microarquitectónicas de eficiencia</u>	9
<u>4.1.4</u>	<u>Espacios de ejecución: user-space y kernel-space</u>	10
<u>4.1.5</u>	<u>Interfaces estándar de observabilidad e instrumentación: perf, RAPL y NVML</u>	12
<u>4.1.6</u>	<u>Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y governors</u>	15
<u>4.1.6.1</u>	<u>Particularidades de DVFS en GPU</u>	18
<u>4.1.7</u>	<u>Cargas de trabajo: kernels, benchmarks y microbenchmarks</u>	19
<u>4.1.7.1</u>	<u>Kernel como unidad de ejecución</u>	20
<u>4.1.7.2</u>	<u>Benchmark y microbenchmark</u>	20
<u>4.1.8</u>	<u>Aprendizaje automático supervisado ligero</u>	21
<u>4.1.8.1</u>	<u>Tipos de modelos ligeros de clasificación supervisada</u>	21
<u>4.1.8.1.1</u>	<u>Modelo predictivo clasificador: Bósques Aleatorios (Random Forest)</u>	22
<u>4.1.8.1.2</u>	<u>Modelo predictivo clasificador: Regresión Logística (Logistic Regression)</u>	23
<u>4.1.8.1.3</u>	<u>Modelo predictivo clasificador: XGBoost (eXtreme Gradient Boosting)</u>	23
<u>4.1.9</u>	<u>Producto Energía–Retardo (EDP)</u>	25
<u>4.2</u>	<u>Estado del Arte</u>	25
<u>5</u>	<u>Metodología</u>	28
<u>5.1</u>	<u>Fase 1: Recolección de información y caracterización de cargas</u>	29

5.2 Fase 2: Desarrollo del modelo de aprendizaje automático	30
5.3 Fase 3: Implementación del agente de control DVFS	30
5.4 Fase 4: Validación experimental y análisis de resultados	31
6 Alcance y limitaciones	31
6.1 Alcance	31
6.2 Limitaciones	32
7 Cronograma	34
8 Presupuesto	36
8.1 Recursos Humanos	36
8.2 Recursos Tecnológicos e Infraestructura	36
8.3 Total General	37
9 Bibliografía	37

1 Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como *ondemand* o *schedutil*, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo overhead, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes compute-bound y memory-bound, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica. Con base en esta clasificación, el sistema ajusta dinámicamente la frecuencia de CPU y GPU con el objetivo de optimizar el Producto Energía–Retardo (EDP). Se espera que este enfoque permita mejorar la eficiencia energética en comparación con los gobernadores tradicionales de Linux, manteniendo una degradación controlada del rendimiento y un overhead mínimo asociado a la inferencia.

2 Planteamiento Y Justificación Del Problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [1] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [2].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (compute-bound) y fases donde está restringido por la transferencia de datos desde memoria (memory-bound). Esta distinción es crítica porque, en el régimen memory-bound, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases compute-bound la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [3], [4].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [5], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos como el clúster SC3 de la Universidad Industrial de Santander, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la

sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [1]–[5]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

3 Objetivos

3.1 Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS) en sistemas heterogéneos CPU-GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

3.2 Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (*Perf* y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (daemon) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía-Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

4 Marco De Referencia

4.1 Marco Conceptual

El ajuste dinámico de frecuencia y voltaje para la optimización energética se fundamenta en la relación no lineal entre los parámetros de operación del hardware y la disipación de calor. A continuación, se definen los principios termodinámicos, arquitectónicos y de aprendizaje computacional que rigen el diseño de este agente de control en espacio de usuario.

4.1.1 Termodinámica del Silicio y Potencia CMOS

La disipación de potencia en un circuito integrado moderno, como una CPU o una GPU basada en tecnología CMOS, puede descomponerse analíticamente en dos componentes principales: potencia estática, asociada a corrientes de fuga, y potencia dinámica, originada por la conmutación de cargas capacitivas en las compuertas lógicas. Dado que las técnicas de Escalado Dinámico de Voltaje y Frecuencia (DVFS) actúan sobre la frecuencia de operación y el voltaje de alimentación, su efecto principal recae sobre la potencia dinámica, la cual, para una compuerta CMOS, puede modelarse como:

$$P_{dynamic} = \alpha \cdot C \cdot V^2 \cdot f$$

En donde α es el factor de actividad, C la capacitancia total del nodo de salida, V el voltaje de alimentación y f la frecuencia de operación. En circuitos más complejos, esta relación se extiende al chip completo, interpretando C como la capacitancia efectiva total conmutada y α como un factor de actividad promedio [\[6, eq. \(1\), p. 1210\]](#).

Esta formulación constituye la base física que justifica el uso de técnicas de Escalado Dinámico de Voltaje y Frecuencia (DVFS). En los estados DVFS del hardware, la frecuencia de operación y el voltaje de alimentación se encuentran acoplados, ya que operar a frecuencias más altas suele requerir mayores voltajes para garantizar la integridad temporal y la estabilidad de las señales. En consecuencia, cuando una reducción de frecuencia permite también disminuir el voltaje, la potencia dinámica puede reducirse de forma significativa, debido a su dependencia lineal con la frecuencia y cuadrática con el voltaje. Esta relación constituye la base del compromiso energía–rendimiento en DVFS, ya que la reducción de voltaje y frecuencia disminuye la potencia dinámica, pero su efecto sobre el tiempo de ejecución depende del comportamiento computacional predominante de la aplicación.

4.1.2 Modelo Roofline y regímenes de limitación del rendimiento

El modelo Roofline proporciona un marco conceptual para analizar el rendimiento de una aplicación en función de su intensidad operacional y de los límites físicos del hardware.

En su formulación clásica, el rendimiento alcanzable de un kernel puede expresarse como:

$$P = \min(P_{pico}, I \cdot BW_{pico})$$

Donde P representa el rendimiento alcanzable, P_{pico} el rendimiento pico de cómputo del sistema, I la intensidad operacional y BW_{pico} el ancho de banda pico de memoria [7].

En el modelo original, la intensidad operacional se define como el número de operaciones por byte transferido entre la jerarquía de caché y la memoria principal, lo que permite relacionar directamente el comportamiento del kernel con la capacidad efectiva del subsistema de memoria. A partir de esta relación, el modelo establece una cota superior de rendimiento: si, para una determinada intensidad operacional, el límite viene dado por el término $I \cdot BW_{pico}$, el kernel se encuentra en un régimen limitado por memoria (*memory-bound*); por el contrario, si el límite viene dado por P_{pico} , el kernel se encuentra en un régimen limitado por cómputo (*compute-bound*) [7].

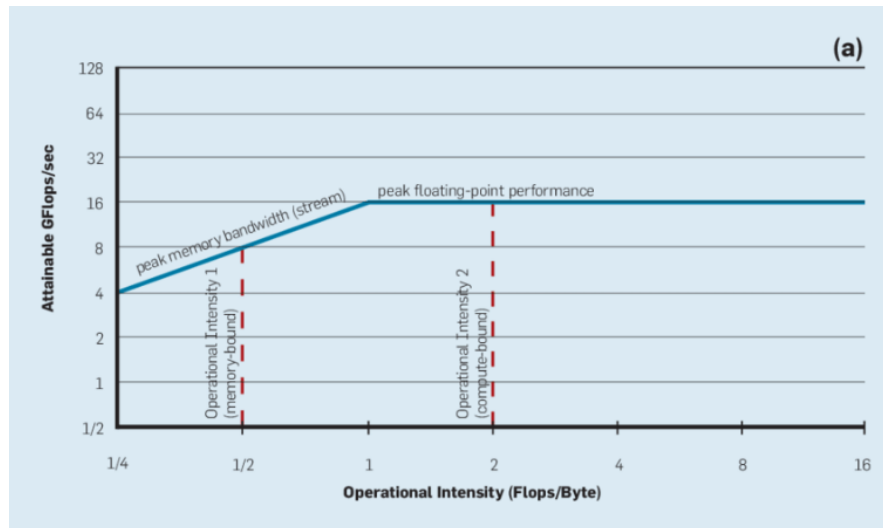


Figura 1: Modelo Roofline conceptual. (a) Relación entre el ancho de banda de memoria y el rendimiento pico de cómputo. Fuente: Tomado de Williams et al. [7, Fig. 1a].

4.1.3 Telemetría microarquitectónica

La aplicación selectiva de DVFS en sistemas heterogéneos requiere mecanismos de observación capaces de caracterizar, con baja intrusión, el comportamiento microarquitectónico de la carga de trabajo durante su ejecución. Para ello, los procesadores modernos incorporan unidades especializadas de monitorización y exponen interfaces estándar que permiten recolectar métricas relacionadas con la ejecución de instrucciones, el acceso a memoria y el consumo de potencia. Estas fuentes

de telemetría constituyen la base empírica para inferir el estado computacional predominante de una aplicación y para cuantificar el efecto energético de las decisiones de control.

4.1.3.1 Unidades de Monitorización de Rendimiento (PMU) y Contadores de Rendimiento por Hardware

Las Unidades de Monitorización de Rendimiento (*Performance Monitoring Units*, PMU) son componentes hardware integrados en los procesadores modernos para observar eventos microarquitectónicos de bajo nivel. Arquitectónicamente, una PMU está compuesta por registros de control y por contadores de monitorización de rendimiento (*Performance Monitoring Counters*, PMCs), los cuales pueden emplearse para registrar eventos asociados al flujo de instrucciones, la jerarquía de memoria y otros aspectos relevantes del comportamiento del procesador [8]. En términos generales, estos contadores se clasifican en dos grupos: contadores fijos, destinados a eventos frecuentes como ciclos de reloj e instrucciones retiradas, y contadores programables o genéricos, configurables para medir distintos eventos del sistema [8].

A través de estos *Hardware Performance Counters* (HPCs), es posible obtener instrumentación de baja sobrecarga y alta resolución para la caracterización de aplicaciones de computación de alto rendimiento. No obstante, aunque los procesadores soportan un conjunto amplio de eventos observables, el número de contadores físicos disponibles para medirlos simultáneamente es limitado, oscilando típicamente entre 1 y 4 PMCs fijos y solo 4 a 8 PMCs genéricos en arquitecturas modernas [8]. Esta restricción condiciona la selección de métricas y obliga a priorizar aquellos eventos más representativos para la caracterización del comportamiento de la aplicación. En consecuencia, las PMU constituyen una fuente de información más rica que las métricas globales tradicionalmente empleadas por los gobernadores del sistema operativo, pero su aprovechamiento exige criterios adecuados de selección y organización de eventos.

4.1.3.2 Métricas microarquitectónicas de eficiencia

La caracterización de los regímenes compute-bound y memory-bound en sistemas reales no suele derivarse de una observación directa del estado interno de la aplicación, sino de señales microarquitectónicas que reflejan cómo se distribuye el costo de ejecución entre el núcleo de cómputo y el subsistema de memoria. En este contexto, métricas como las instrucciones por ciclo (*Instructions Per Cycle*, IPC), la tasa de fallos de caché y los ciclos de estancamiento (*stall cycles*) resultan relevantes porque describen manifestaciones observables de la eficiencia de ejecución y de la presión sobre la jerarquía de memoria [9], [10].

El IPC expresa la cantidad promedio de instrucciones retiradas por ciclo de reloj y mide el grado en que el procesador transforma ciclos de ejecución en trabajo útil. Valores relativamente altos son consistentes con un flujo sostenido de actividad efectiva, mientras que valores bajos sugieren que una mayor fracción del tiempo de ejecución está siendo absorbida por latencias, dependencias o esperas por recursos [\[11, Ch. 6, p. 225\]](#). Por ello, el IPC puede entenderse como una señal del aprovechamiento efectivo del núcleo, aunque no basta por sí solo para caracterizar completamente el régimen de ejecución. De forma complementaria, la tasa de fallos de caché describe la proporción de accesos que no pueden resolverse dentro de la jerarquía de caché y deben continuar hacia niveles de mayor latencia [\[11, Ch. 2, pp. 35-36\]](#). En particular, la tasa de fallos en el último nivel de caché (*Last-Level Cache Miss Rate*, *LLC Miss Rate*) ofrece una señal relevante del peso relativo del acceso a memoria dentro del comportamiento global de la aplicación [\[11, Ch. 6, pp. 231-232\]](#). A diferencia del número absoluto de fallos, una razón o porcentaje de *misses* permite interpretar con mayor claridad la presión real sobre el subsistema de memoria. En este mismo sentido, un *stall* puede definirse como un ciclo o conjunto de ciclos en los que el pipeline no logra avanzar al ritmo esperado porque el procesador debe esperar datos, recursos o la resolución de dependencias internas. Cuando estos estancamientos se asocian predominantemente a la latencia de memoria, indican que una mayor fracción del tiempo de ejecución está condicionada por la llegada de datos y no por la capacidad aritmética del núcleo [\[11, Ch. 6, p. 223\]](#).

En conjunto, estas métricas no constituyen una formulación analítica exacta de los estados compute-bound y memory-bound, pero sí ofrecen una representación observable de aspectos complementarios del comportamiento de ejecución [\[9\]](#), [\[10\]](#). El IPC refleja el aprovechamiento efectivo del núcleo, la tasa de fallos de caché la presión relativa sobre la jerarquía de memoria y los *stall cycles* las interrupciones en la continuidad del trabajo útil. Por ello, su valor teórico radica en que permiten describir patrones de ejecución consistentes con distintos regímenes de limitación del rendimiento [\[10\]](#).

4.1.4 Espacios de ejecución: *user-space* y *kernel-space*

Los sistemas operativos modernos organizan la ejecución del software en dominios de privilegio diferenciados, comúnmente denominados *user-space* y *kernel-space*. Esta separación se apoya en la arquitectura del procesador, que permite operar al menos en modo usuario y modo kernel. En modo usuario, el software solo puede acceder a memoria marcada para el espacio de usuario; cualquier intento de acceso a memoria del kernel produce una excepción de hardware. En modo kernel, en cambio, el procesador puede acceder tanto al espacio del kernel como al del usuario [\[12, ch. 2\]](#).

Esta distinción constituye un mecanismo fundamental de protección y control. Operaciones sensibles como la gestión de memoria, el acceso directo al hardware o las operaciones de entrada y salida requieren privilegios de kernel, lo que impide que los procesos ordinarios interfieran con estructuras críticas del sistema [12, ch. 2]. En este marco, *kernel-space* designa el dominio donde residen el núcleo del sistema operativo y los mecanismos que arbitran el acceso a los recursos físicos, mientras que *user-space* comprende programas, bibliotecas y entornos de ejecución que operan fuera de ese núcleo [13]. Para el software de aplicación, esto implica que el acceso a recursos privilegiados no ocurre de forma directa, sino mediante mecanismos de mediación del sistema operativo. En sistemas tipo Unix y Linux, esta mediación se realiza principalmente a través de las *system calls*, que constituyen la interfaz entre los programas de *user-space* y los servicios del kernel [13]. Así, cuando una aplicación necesita abrir un archivo, reservar memoria o interactuar con dispositivos y contadores del sistema, no actúa directamente sobre el hardware, sino que solicita al kernel la operación correspondiente dentro de un marco de control y aislamiento [12, ch. 2], [13].

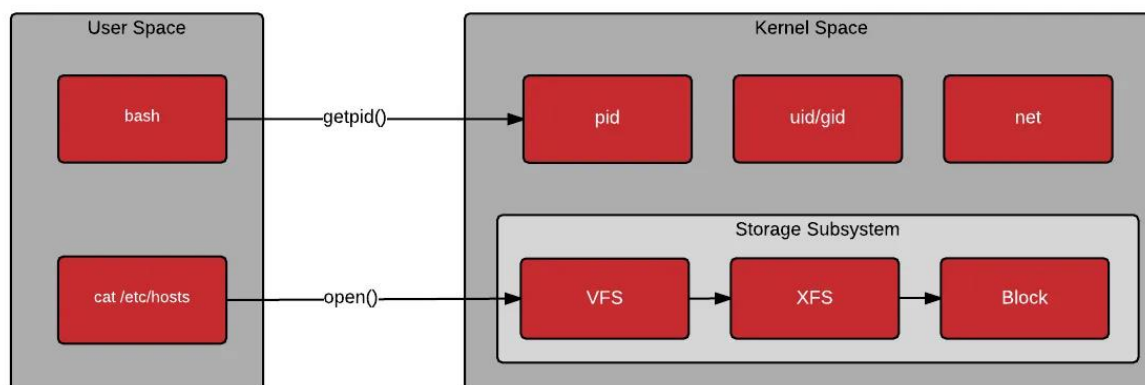


Figura 2: Relación conceptual entre *user-space*, *system calls* y *kernel-space*. Fuente: Tomado de McCarty [13].

Esta distinción resulta especialmente relevante en problemas de observabilidad y gestión del hardware. Dado que los programas en *user-space* no poseen acceso irrestricto a dispositivos ni a estructuras internas del sistema, cualquier mecanismo de monitoreo o control implementado fuera del kernel debe apoyarse en interfaces explícitamente expuestas por la plataforma. Por ello, la separación entre *user-space* y *kernel-space* no solo define un modelo de protección del sistema operativo, sino también el alcance y las restricciones de las soluciones software que buscan observar el comportamiento del hardware o influir sobre su estado operativo [12, ch. 2]; [13].

4.1.5 Interfaces estándar de observabilidad e instrumentación: perf, RAPL y NVML

La observabilidad del comportamiento de sistemas heterogéneos CPU–GPU depende de interfaces que permitan acceder, desde software de alto nivel, a métricas internas de rendimiento, utilización y consumo energético sin requerir la modificación directa del kernel ni de los controladores del hardware. En este contexto, las plataformas modernas exponen mecanismos estándar que posibilitan la recolección de señales relevantes para el análisis del sistema. Entre ellas, destacan la interfaz *perf_event* en Linux para contadores de rendimiento del procesador, Intel RAPL para dominios energéticos de CPU, y la biblioteca NVIDIA Management Library (NVML) para telemetría operativa y energética de GPUs NVIDIA [4], [14], [15], [16].

En sistemas Linux, *perf* constituye una de las interfaces estándar más utilizadas para acceder a la PMU del procesador. A través de la infraestructura *perf_event*, el kernel expone eventos de bajo nivel asociados a ciclos de reloj, instrucciones retiradas, fallos de caché y otros indicadores microarquitectónicos del comportamiento del núcleo [14]. En esencia, *perf* representa el puente entre los contadores de rendimiento hardware y su observación desde *user-space*, permitiendo instrumentar el comportamiento del procesador mediante una interfaz homogénea y ampliamente adoptada en Linux [14].

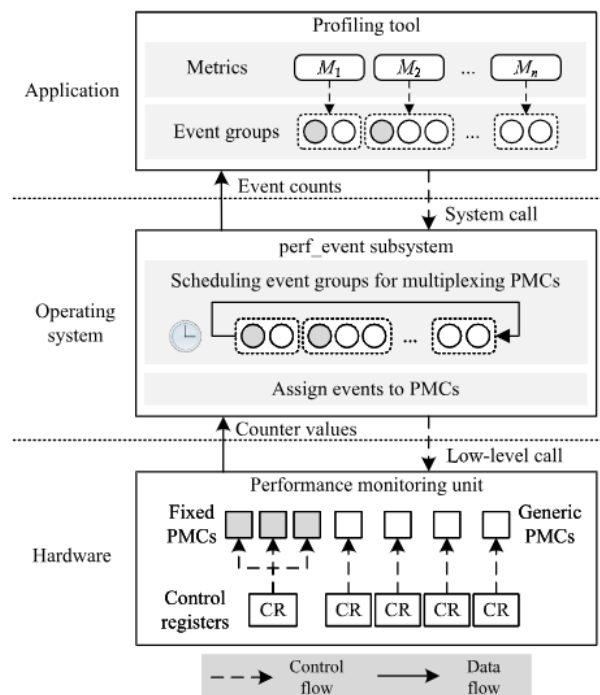


Figura 3: Workflow para recopilar datos de rendimiento de la microarquitectura basados en *perf_event* de Linux. Fuente: Tomado de T.-Y. Liu et al. [8].

De manera complementaria, Intel RAPL (*Running Average Power Limit*) proporciona una interfaz integrada en muchos procesadores Intel para observar el comportamiento energético del sistema a través de distintos dominios internos. Introducido originalmente como un mecanismo de modelado energético y posteriormente refinado en microarquitecturas más recientes, RAPL expone información de energía acumulada para dominios como el paquete del procesador (*Package*), los núcleos (*Core* o *Power Plane 0*), la memoria DRAM y, cuando existe, otros componentes específicos del sistema [15], [17]. Esta organización por dominios resulta conceptualmente relevante porque permite desagregar parcialmente el comportamiento energético del procesador y de la memoria principal, lo que facilita el análisis de la relación entre consumo energético y configuración operativa del hardware.

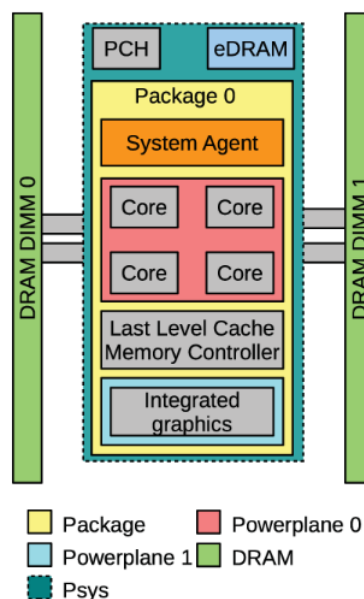


Figura 4: Descripción general de los componentes del sistema que abarca cada dominio RAPL. Fuente: Tomado de P. Thamm and U. Leser [17, Fig. 2].

RAPL reporta energía mediante registros específicos del modelo (*Model Specific Registers*, MSR), cuyos valores representan energía acumulada en unidades dependientes de la plataforma [17]. Estas lecturas no cubren de forma exhaustiva el consumo total del nodo y la disponibilidad de dominios depende de la microarquitectura y del modelo de CPU [17]. Además, al ser contadores acumulativos, su interpretación debe considerar la periodicidad de actualización y el posible desbordamiento de los registros [17]. Aun así, RAPL es una de las interfaces más utilizadas en estudios de eficiencia energética y DVFS sobre CPU, porque ofrece una vía estandarizada y de baja intrusión para observar variaciones relativas de energía y potencia sin instrumentación externa especializada [4], [15], [17].

En el caso del acelerador gráfico, la interfaz estándar más extendida en plataformas NVIDIA es la *NVIDIA Management Library* (NVML). Esta biblioteca permite consultar información relacionada con el estado operativo de la GPU, incluyendo métricas de utilización, potencia, memoria y otros parámetros relevantes del dispositivo [16]. A diferencia de *perf*, cuyo foco principal es la observación de eventos microarquitectónicos del procesador, NVML se orienta a la telemetría y gestión del acelerador como dispositivo completo, ofreciendo una vista del comportamiento de la GPU útil tanto para monitoreo como para análisis energético [4], [16].

En conjunto, *perf*, RAPL y NVML no cumplen funciones idénticas, pero sí conforman una base complementaria de observabilidad para sistemas heterogéneos. *perf* permite acceder a señales del comportamiento microarquitectónico del procesador; RAPL aporta una vía de observación energética de la CPU; y NVML extiende esta capacidad al dominio de la GPU. Por ello, estas interfaces constituyen un fundamento conceptual importante para el estudio de mecanismos de monitoreo y gestión energética en entornos HPC, al proporcionar acceso estandarizado a métricas relevantes de rendimiento y consumo sin necesidad de intervenir directamente las capas internas del sistema [4], [14], [15], [16].

4.1.6 Gestión dinámica de frecuencia en sistemas operativos: DVFS, *P-states* y *governors*

La gestión dinámica de voltaje y frecuencia (*Dynamic Voltage and Frequency Scaling*, DVFS) constituye uno de los mecanismos centrales de control energético en procesadores modernos. Su propósito general es ajustar el punto operativo del hardware mediante cambios coordinados en la frecuencia de reloj y el voltaje de alimentación, con el fin de reducir consumo y disipación térmica cuando la carga de trabajo no requiere el máximo desempeño disponible. En arquitecturas contemporáneas, este mecanismo no depende únicamente de una relación física entre voltaje y frecuencia, sino también de recursos hardware dedicados a la gestión de potencia, entre ellos reguladores de voltaje y unidades de control de potencia (*Power Control Units*, PCUs o *P-Units*), que supervisan dominios funcionales del procesador y ejecutan solicitudes de cambio de estado operativo [18]. Desde la dimensión del sistema operativo, DVFS se expresa a través de abstracciones de estado que desacoplan la política software de los detalles eléctricos internos del procesador. En particular, el estándar ACPI (*Advanced Configuration and Power Interface*) mencionado en [11] y [18] define los *performance states* o *P-states*, que representan distintos niveles operativos de frecuencia y voltaje dentro del estado activo del procesador (C0), y los *C-states*, que representan niveles progresivos de inactividad o reposo. En este esquema, el estado P0 corresponde al mayor nivel de rendimiento disponible, mientras que estados superiores en numeración (P1, P2, ..., Pn) representan frecuencias progresivamente menores; de forma análoga, los *C-states* más profundos

permiten mayores ahorros de energía a costa de mayores latencias de reactivación. Así, los *P-states* permiten modular el rendimiento de componentes activos, mientras que los *C-states* gestionan la inactividad de componentes o dominios no utilizados [18], [11].

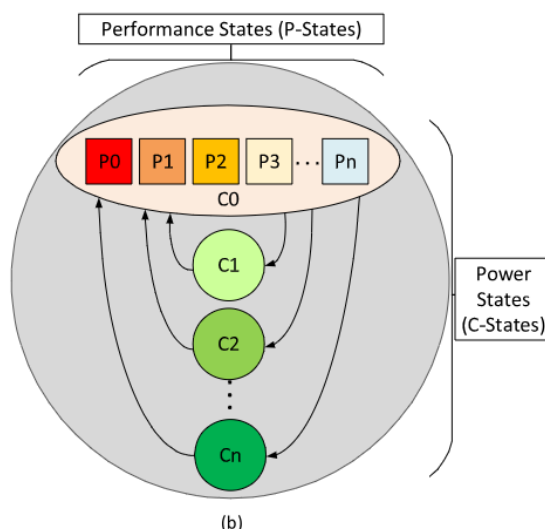


Figura 5: *Processor power (C-states) y performance states (P-states).* Fuente: Tomado de R. Hebbar and A. Milenković [18, Fig. 1b].

Sobre esta base operan los *governors*, es decir, políticas que deciden cuándo y cómo cambiar el estado operativo del procesador. Un *governor* no implementa el mecanismo físico de DVFS, sino la lógica de control que interpreta señales del sistema y solicita transiciones entre *P-states*. En términos generales, estas políticas pueden agruparse en gobernadores de frecuencia fija y gobernadores dinámicos basados en DVFS [18]. Las políticas de frecuencia fija sitúan al procesador en extremos operativos. *performance* mantiene de forma estática la frecuencia máxima, lo que favorece cargas sensibles a latencia, pero puede resultar energéticamente ineficiente en escenarios de baja utilización. En el extremo opuesto, *powersave* fija la frecuencia mínima, reduciendo la potencia instantánea a costa de un mayor tiempo de ejecución; en consecuencia, este menor consumo no siempre implica menor energía total [18].

Para evitar estas dos situaciones extremas, Linux incorpora gobernadores dinámicos basados en DVFS. *ondemand* ajusta la frecuencia según la carga media del procesador, elevándola cuando la utilización supera un umbral y reduciéndola cuando la carga disminuye. *conservative* sigue una lógica similar, pero realiza cambios más graduales. Por su parte, *schedutil* integra la decisión de frecuencia con el planificador del sistema operativo. Aunque su comportamiento general es comparable al de *ondemand*, este último suele ofrecer mayor flexibilidad de ajuste [18].

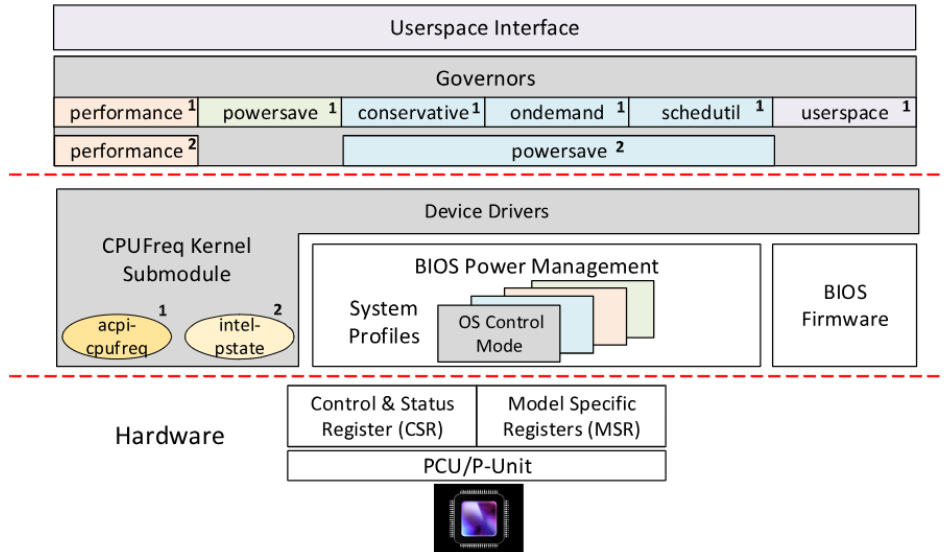


Figura 5: Jerarquía de componentes de gestión de energía. Fuente: Tomado de R. Hebbar and A. Milenković [18, Fig. 2].

En Linux, esta política no actúa de forma aislada, sino sobre una jerarquía de componentes que va desde el hardware y sus registros de control hasta los drivers del kernel y la interfaz expuesta al espacio de usuario, como se puede visualizar en la figura 5. El trabajo de Hebbar y Milenković [18] resume esta jerarquía mostrando que el control de potencia involucra el hardware de gestión energética, la BIOS, los drivers de transición de estado y, finalmente, los *governors* accesibles desde la capa de usuario. Dentro de este esquema, distinguen dos controladores principales: *ACPI-CPUFreq*, desarrollado por la comunidad de Linux y utilizado de forma genérica en múltiples plataformas, e *Intel P-state*, empleado como controlador por defecto en muchas generaciones de procesadores Intel desde la arquitectura *Sandy Bridge*. Ambos actúan como intermediarios entre la lógica de control del sistema operativo y la infraestructura interna de gestión de potencia del procesador. La diferencia entre ambos controladores no es menor, bajo *acpi-cpufreq*, Linux expone una familia de *governors* genéricos, entre ellos *performance*, *powersave*, *ondemand*, *conservative*, *schedutil* y *userspace*. En cambio, bajo *intel-pstate* el modelo cambia, ya que solo se soportan dos *governors*: *performance* y *powersave*, y que este último asume el papel de política dinámica equivalente, en términos funcionales, a enfoques como *ondemand* o *schedutil*. Además, *intel-pstate* puede aprovechar los llamados *hardware P-states (HWP)*, en los cuales el procesador selecciona autónomamente los *P-states* a partir de la utilización observada, con menor intervención directa del sistema operativo [18]. Este punto es conceptualmente relevante porque muestra que la gestión de frecuencia en sistemas modernos no sigue un único modelo universal, sino que depende del grado de autonomía transferido al hardware y del acoplamiento entre firmware, kernel y microarquitectura.

Otro componente fundamental de este problema es la latencia de conmutación de frecuencia. Las transiciones entre *P-states* no son instantáneas: implican un costo temporal asociado al procesamiento de la solicitud y al ajuste efectivo de frecuencia y voltaje. Según lo propuesto en [18] estas transiciones pueden tomar aproximadamente 10 ms, mientras que con HWP pueden reducirse a alrededor de 1 ms. Aunque estos valores dependen de la plataforma, la implicación conceptual es general: cualquier política de control que pretenda modificar frecuencias de manera continua debe considerar que el costo de transición puede volverse relevante si las decisiones se toman con excesiva frecuencia o si las fases de ejecución cambian más rápido que la capacidad efectiva del sistema para materializar el nuevo estado.

En conjunto, DVFS, los *P-states*, los *governors* y los controladores de frecuencia del sistema operativo conforman el marco operativo dentro del cual se implementa la gestión energética en procesadores modernos.

4.1.6.1 Particularidades de DVFS en GPU

En GPU, la gestión dinámica de voltaje y frecuencia no puede interpretarse exactamente bajo el mismo esquema conceptual que en CPU. Mientras en procesadores convencionales la discusión sobre DVFS suele centrarse en estados de rendimiento asociados al procesador y a sus núcleos, en aceleradores gráficos modernos el comportamiento energético y temporal de una aplicación depende de la interacción entre múltiples dominios funcionales, en particular el dominio de cómputo y el dominio de memoria. De forma general, las arquitecturas GPU modernas distinguen al menos un dominio asociado al procesamiento, que agrupa los multiprocesadores de *streaming* (*Stream Multiprocessors*, *SM*) y parte de la jerarquía interna de caché, y un dominio asociado a la memoria principal del dispositivo. Esta separación implica que cambios en la frecuencia del núcleo y cambios en la frecuencia de memoria no producen necesariamente el mismo efecto sobre el tiempo de ejecución ni sobre la potencia consumida. Por ello, la respuesta de una aplicación a DVFS en GPU no depende únicamente de la capacidad de cómputo disponible, sino también de la presión ejercida sobre el subsistema de memoria y de la relación entre ambos dominios [19].

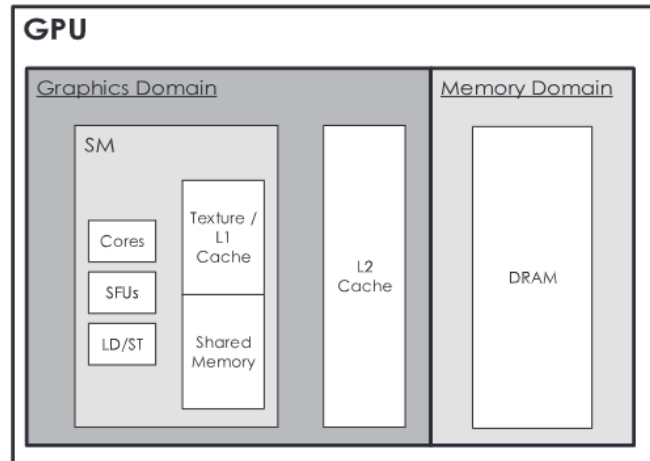


Figura 6: Dominios de frecuencia existentes en dispositivos GPU modernos (e.g. NVIDIA Kepler, Maxwell and Pascal GPUs). Fuente: Tomado de J. Guerreiro et al. [19, Fig. 1].

La distinción entre cargas *compute-bound* y *memory-bound* es especialmente importante en GPU. Cuando el rendimiento está limitado por el cómputo, la frecuencia del dominio de procesamiento impacta más directamente el tiempo de ejecución. En cambio, cuando predomina la limitación por memoria, la sensibilidad del rendimiento frente a la frecuencia del núcleo disminuye y el dominio de memoria pasa a ser más determinante. Por ello, el efecto de DVFS en GPU debe analizarse según la relación entre cómputo y memoria, y no solo como una variación de la frecuencia del núcleo [19], [20]. Además, en GPU la potencia, el rendimiento y la energía no varían de forma independiente con la frecuencia. Reducir la frecuencia puede bajar la potencia instantánea, pero no necesariamente la energía total si el tiempo de ejecución aumenta demasiado; de forma análoga, subir la frecuencia puede mejorar el rendimiento sin traducirse en una mejora proporcional de la eficiencia energética. Por tanto, el análisis de DVFS debe considerar conjuntamente tiempo de ejecución y consumo de potencia, especialmente cuando existen dominios de frecuencia diferenciados [19], [20].

En síntesis, la especificidad de DVFS en GPU radica en que el problema de control no recae sobre un único dominio homogéneo, sino sobre una arquitectura donde coexisten subsistemas con sensibilidades distintas frente a la frecuencia. Esta característica diferencia conceptualmente la gestión de frecuencia en GPU respecto de la CPU y justifica que su análisis teórico incorpore explícitamente la relación entre cómputo, memoria y consumo energético [19].

4.1.7 Cargas de trabajo: *kernels*, *benchmarks* y *microbenchmarks*

En computación de alto rendimiento, el análisis del comportamiento de una aplicación rara vez se realiza únicamente a nivel del programa completo. En su lugar, resulta

habitual estudiar unidades de ejecución más acotadas, patrones de carga bien definidos y conjuntos estandarizados de pruebas que permitan observar de manera controlada el rendimiento, el uso de recursos y la sensibilidad del sistema frente a distintos mecanismos de gestión. En este contexto, los conceptos de *kernel*, *benchmark* y *microbenchmark* constituyen herramientas fundamentales para describir y comparar el comportamiento computacional de cargas de trabajo científicas y de propósito general.

4.1.7.1 Kernel como unidad de ejecución

En el ámbito de HPC, el término *kernel* no se refiere al núcleo del sistema operativo, sino a una rutina computacional bien delimitada que concentra una fracción relevante del trabajo de una aplicación. Un kernel suele representar una operación dominante dentro del flujo de ejecución, como una multiplicación de matrices, una transformación rápida de Fourier, una actualización de *stencil* o un recorrido estructurado sobre memoria. Debido a que estas rutinas encapsulan patrones recurrentes de acceso a datos y de uso de unidades aritméticas, su análisis permite estudiar con mayor precisión el comportamiento de una aplicación que si se considera únicamente el programa completo [\[21\]](#). Bajo un enfoque de rendimiento, el kernel constituye una unidad útil de observación porque suele exhibir una firma computacional más estable que la aplicación global. Mientras una aplicación completa puede combinar distintas fases con demandas heterogéneas sobre cómputo, memoria y comunicación, un kernel tiende a concentrar un patrón dominante de ejecución. Por ello, el estudio de kernels resulta especialmente pertinente para analizar regímenes *compute-bound* y *memory-bound*, ya que permite aislar con mayor claridad la relación entre intensidad computacional, uso de memoria y sensibilidad a cambios en la frecuencia de operación.

4.1.7.2 Benchmark y microbenchmark

Un *benchmark* es una carga de trabajo diseñada o seleccionada con el propósito de medir y comparar el comportamiento de un sistema bajo condiciones reproducibles. En el contexto de HPC, los *benchmarks* pueden representar aplicaciones completas, miniaplicaciones o *kernels* representativos de dominios científicos concretos. Su valor radica en que permiten evaluar rendimiento, consumo energético o escalabilidad sobre una base común, facilitando la comparación entre arquitecturas, configuraciones o políticas de control [\[11, Ch. 12\]](#). Dentro de esta categoría, un *microbenchmark* constituye una forma más reducida y controlada de *benchmark*, orientada a aislar el comportamiento de un componente, mecanismo o patrón específico. Un *microbenchmark* no pretende reproducir toda la complejidad de una aplicación real, sino capturar de manera precisa una propiedad particular del sistema, por ejemplo, el rendimiento de una operación aritmética intensiva, el ancho de banda de memoria o la latencia asociada a un acceso repetitivo [\[11, Ch. 12\]](#). En consecuencia, mientras los *benchmarks* ofrecen una visión más cercana al comportamiento de cargas reales, los

microbenchmarks permiten estudiar fenómenos de bajo nivel con menor interferencia de factores externos.

4.1.8 Aprendizaje automático supervisado ligero

El aprendizaje supervisado es un paradigma del aprendizaje automático cuyo objetivo es inferir una función que mapea un conjunto de variables de entrada hacia una variable de salida a partir de datos de entrenamiento previamente etiquetados. En este enfoque, el modelo aprende regularidades a partir de ejemplos históricos en los que la clase correcta ya es conocida, con el propósito de generalizar ese conocimiento sobre observaciones no vistas. De manera general, el conjunto de datos se divide en subconjuntos de entrenamiento y prueba, de modo que el primero se utiliza para ajustar el modelo y el segundo para examinar su capacidad de generalización fuera de los ejemplos usados durante el aprendizaje [\[22\]](#).

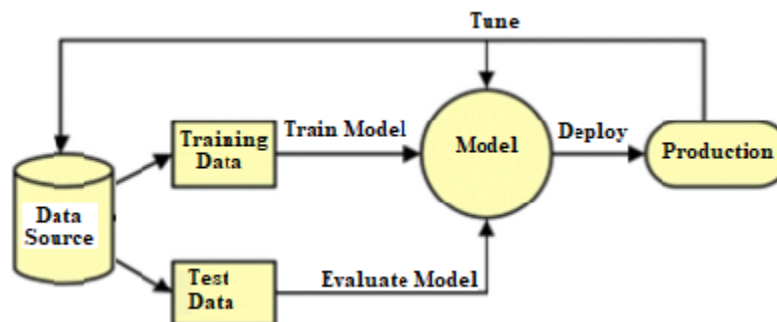


Figura 7: Workflow del Aprendizaje Supervisado. Fuente: Tomado de D. Pandey. [\[22, Fig. 1\]](#).

4.1.8.1 Tipos de modelos ligeros de clasificación supervisada

En contextos donde la decisión debe ejecutarse con restricciones de tiempo y recursos, resulta especialmente relevante la noción de modelo ligero. En términos generales, un clasificador ligero es aquel cuya inferencia puede realizarse con baja complejidad computacional y consumo acotado de memoria, manteniendo al mismo tiempo una capacidad razonable de generalización. Esta ligereza no depende solo del nombre del algoritmo, sino también de la complejidad efectiva del modelo entrenado y del costo aceptable de predicción en el entorno donde será utilizado. Entre los clasificadores supervisados de uso más extendido se encuentran los árboles de decisión, los bosques aleatorios, la regresión logística, las máquinas de soporte vectorial y los métodos basados en vecinos cercanos como *K-nearest neighbors* [\[22\]](#). Cada uno de ellos impone un criterio distinto de separación entre clases y presenta compromisos diferentes entre interpretabilidad, flexibilidad, robustez y costo de inferencia.

4.1.8.1.1 Modelo predictivo clasificador: Bósques Aleatorios (*Random Forest*)

El algoritmo de Bosques Aleatorios (*Random Forest*) es un método de aprendizaje automático supervisado fundamentado en el paradigma del aprendizaje en conjunto (*ensemble learning*). Este enfoque se basa en el principio de que la combinación de múltiples modelos predictivos base puede resolver problemas computacionales complejos para generar una predicción final mucho más robusta y precisa que la de un modelo individual [23]. En esta arquitectura específica, los modelos base utilizados son los árboles de decisión. Durante la fase de entrenamiento, el algoritmo construye un bosque compuesto por una multitud de estos árboles de forma independiente. Cuando el modelo se enfrenta a datos no observados, cada árbol individual emite su propia predicción y el algoritmo agrega estos resultados para tomar una decisión final; en tareas de clasificación, la salida es la clase con la votación mayoritaria (*majority vote*) [23]. Para garantizar que los árboles no sean idénticos y aporten diversidad al conjunto, el modelo implementa dos mecanismos estocásticos clave. El primero es la agregación de *bootstrap* (*bagging*), mediante el cual cada árbol se entrena utilizando una muestra aleatoria diferente extraída del conjunto de entrenamiento original con reemplazo [23]. El segundo mecanismo es la aleatoriedad de características (*feature randomness*), que establece que, al construir la división (*split*) en cada nodo del árbol, no se evalúan todas las variables disponibles, sino un subconjunto aleatorio de estas [23].

La combinación de ambos mecanismos garantiza una baja correlación entre los árboles individuales, lo que mitiga directamente el problema de sobreajuste (*overfitting*) que suelen sufrir los árboles de decisión profundos [23].

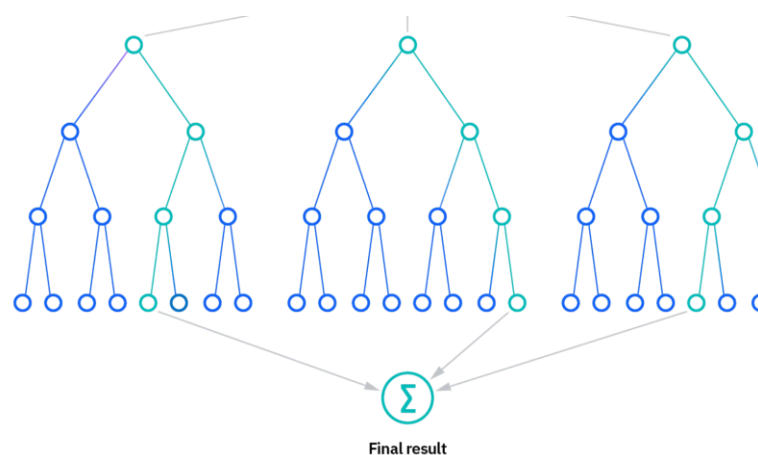


Figura 8: Esquema de un *Random Forest*. Fuente: Tomado de IBM [23].

4.1.8.1.2 Modelo predictivo clasificador: Regresión Logística (*Logistic Regression*)

La regresión logística es un algoritmo de clasificación supervisada utilizado para predecir una salida categórica. A diferencia de la regresión lineal, que modela variables continuas, la regresión logística estima la probabilidad de pertenencia a una clase, por lo que su salida queda acotada en el intervalo $[0,1]$. En su forma más común, la regresión logística binaria modela eventos dicotómicos, es decir, problemas con dos posibles clases [24].

Su formulación parte de un predictor lineal de la forma:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Donde $x_1, x_2, \dots, x_n$ son las variables de entrada y $\beta_0, \beta_1, \dots, \beta_n$ son los coeficientes del modelo. Como este predictor lineal puede tomar valores en $(-\infty, \infty)$, no puede interpretarse directamente como probabilidad [24]. Para resolverlo, la regresión logística aplica una transformación sigmoide, que restringe la salida al intervalo $[0,1]$:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-z}}$$

De este modo, la salida del modelo puede interpretarse como la probabilidad de que una observación pertenezca a la clase positiva. Conceptualmente, sus principales ventajas son la simplicidad, el bajo costo de inferencia y la facilidad de interpretación, mientras que su principal limitación es que induce una frontera de decisión lineal, por lo que puede resultar insuficiente cuando la separación entre clases depende de relaciones fuertemente no lineales [24].

4.1.8.1.3 Modelo predictivo clasificador: *XGBoost (eXtreme Gradient Boosting)*

XGBoost (eXtreme Gradient Boosting) es un algoritmo supervisado de clasificación y regresión basado en *ensambles* de árboles de decisión potenciados secuencialmente. Su principio general consiste en construir múltiples árboles débiles de manera aditiva, de modo que cada nuevo árbol aprenda a corregir los errores cometidos por los anteriores. En este sentido, *XGBoost* pertenece a la familia de los *gradient boosted decision trees*, donde el proceso de mejora del modelo se guía por la minimización iterativa de una función de pérdida mediante descenso de gradiente. A diferencia de un árbol de decisión individual, que puede presentar alta varianza y tendencia al sobreajuste, *XGBoost* incrementa la robustez predictiva al combinar secuencialmente múltiples árboles. Cada iteración parte de los residuos o errores del modelo actual y entrena un nuevo árbol que aproxime la corrección necesaria. De esta forma, el clasificador final se obtiene como la suma de contribuciones de árboles sucesivos, lo que le permite capturar relaciones no lineales e interacciones complejas entre variables de entrada [25].

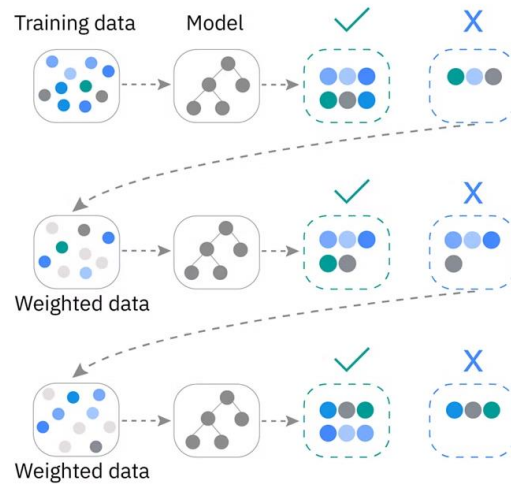


Figura 9: Esquema conceptual del entrenamiento secuencial en *XGBoost*. Fuente: Tomado de IBM [25].

Una característica distintiva de *XGBoost* frente a implementaciones más simples de *gradient boosting* es que incorpora regularización directamente en el objetivo de aprendizaje, lo que ayuda a controlar la complejidad del modelo y a mitigar el sobreajuste. Además, la biblioteca está diseñada para ofrecer alta eficiencia computacional, con soporte para procesamiento paralelo y distribuido, mecanismos de optimización orientados a caché y tratamiento explícito de valores faltantes. Estas propiedades explican su amplia adopción en problemas de clasificación tabular donde se requiere un equilibrio entre capacidad predictiva, escalabilidad y costo de inferencia razonable [25]. Atendiendo a su naturaleza funcional, *XGBoost* puede entenderse como un modelo supervisado de clasificación más expresivo que la regresión logística y más robusto que un árbol individual, aunque a costa de una mayor complejidad estructural. Por ello, resulta especialmente relevante cuando la separación entre clases depende de relaciones no lineales entre *features* y cuando se busca mejorar la precisión sin recurrir necesariamente a modelos de aprendizaje profundo.

4.1.9 Producto Energía–Retardo (EDP)

La evaluación de estrategias de optimización energética en sistemas de alto rendimiento no puede basarse únicamente en minimizar la energía consumida o el tiempo de ejecución por separado. Una política que reduzca el consumo energético a costa de una degradación severa del rendimiento puede resultar globalmente inconveniente; de forma análoga, una configuración orientada exclusivamente al máximo desempeño puede implicar un costo energético desproporcionado. En este contexto, el Producto Energía–Retardo (*Energy–Delay Product*, EDP) se emplea como una métrica fusionada que integra ambas dimensiones en una sola magnitud [26].

De forma general, esta familia de métricas puede expresarse como:

$$EDP = E \cdot T^w$$

En donde E representa la energía consumida durante la ejecución, T el tiempo de ejecución, y w un factor de ponderación que permite ajustar la importancia relativa del retardo dentro de la métrica [26]. Cuando $w = 1$, se obtiene el EDP *clásico*; cuando $w = 2$, se obtiene una variante que penaliza más fuertemente la degradación del tiempo de ejecución, comúnmente asociada al *Energy–Delay² Product* (ED2P) [5], [26].

La utilidad del EDP radica en que evita decisiones sesgadas por una sola dimensión de análisis. En [26] se remarca precisamente que se trata de una métrica construida para observar simultáneamente múltiples criterios en una sola función, mientras que en [5] la emplean como mecanismo para seleccionar configuraciones DVFS que logren un equilibrio entre ahorro energético y degradación del desempeño. En esa misma línea, se advierte que reducir únicamente la potencia puede inducir pérdidas de rendimiento que terminen aumentando la energía consumida, razón por la cual una función multiobjetivo basada en EDP resulta más adecuada para este tipo de problemas [5].

4.2 Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) y las técnicas afines de *frequency capping* y *power capping* se han consolidado como mecanismos de control relevantes tanto en procesadores como en aceleradores GPU. Sin embargo, la eficacia de estos mecanismos depende de forma crítica del comportamiento dinámico de la carga de trabajo, lo que ha impulsado el desarrollo de modelos de caracterización, predicción y control cada vez más sofisticados [1], [2], [4], [5], [19].

Una primera línea de trabajo corresponde a la evaluación empírica del impacto de DVFS sobre aplicaciones y *kernels* HPC. Calore et al. analizaron técnicas DVFS sobre procesadores y aceleradores modernos desde el punto de vista del usuario, mostrando que el beneficio energético no es uniforme y que depende del carácter compute-bound o memory-bound de las partes críticas de la aplicación. Este resultado es relevante porque confirma que la optimización energética no puede formularse como una política global única, sino como una decisión sensible al perfil de ejecución de cada kernel o fase [4]. En una línea similar, Simsek et al. estudiaron simulaciones astrofísicas sobre GPU y mostraron que la instrumentación del código y el ajuste estático y dinámico de frecuencia permiten reducir el consumo energético con pérdidas moderadas de rendimiento,

reportando reducciones de energía por GPU de hasta 7.82% con una pérdida de rendimiento de 2.95% [1]. Más recientemente, Costa et al. evaluaron *frequency capping* y *power capping* en un sistema exascale, observando que la efectividad relativa de cada mecanismo depende tanto de la naturaleza de la aplicación como de la escala de ejecución; en particular, *frequency capping* tendió a ofrecer mejores resultados energéticos a escala, mientras *power capping* resultó más adecuado en ciertos escenarios de nodo único o utilización irregular [2].

Una segunda línea de investigación se centra en los métodos offline de selección de frecuencia óptima. En este grupo destaca el trabajo de Guerreiro et al., quienes propusieron un esquema de clasificación de aplicaciones GPGPU consciente de DVFS, capaz de inferir, a partir de eventos de hardware recolectados a una sola frecuencia, cómo cambiarán tiempo de ejecución, potencia y energía al recorrer el resto del espacio de frecuencias. Su propuesta permitió encontrar pares de frecuencias casi óptimos, con ahorros medios de energía de 16% a 20% y picos de hasta 36%, según la arquitectura evaluada [19]. De forma complementaria, Ali et al. desarrollaron un método automatizado y portable para seleccionar la frecuencia óptima de GPU a partir de tres etapas: caracterización de *features*, modelado analítico de potencia y tiempo de ejecución, y selección multiobjetivo mediante EDP y ED2P. Su metodología requiere recolectar métricas en una configuración base y luego estimar el comportamiento en el resto del espacio DVFS, alcanzando ahorros promedio de 29.6% de energía con 5.2% de pérdida de rendimiento en GA100 y 22.6% con 4.7% en GV100 [5]. Estos trabajos constituyen antecedentes sólidos porque muestran que el espacio DVFS puede explorarse sin fuerza bruta y con buena portabilidad. No obstante, comparten una limitación importante: su lógica de decisión es esencialmente *offline* o precomputada, por lo que no aborda de manera explícita la adaptación fina a la multifasicidad intra-ejecución de aplicaciones HPC generales.

Una tercera línea aborda la caracterización y clasificación de cargas o *kernels* a partir de telemetría de bajo nivel. Shekoffte et al. demostraron que la clasificación de *kernels* GPU puede realizarse con un conjunto reducido de métricas, seleccionadas para minimizar overhead sin sacrificar precisión, lo que confirma que la identificación online de comportamientos como compute-bound y memory-bound es técnicamente viable. Sin embargo, su objetivo principal fue mejorar la planificación y co-ejecución de kernels, no gobernar DVFS [9]. De manera análoga, Littman y Deakin exploraron el uso de aprendizaje automático para clasificar límites de rendimiento a partir de *performance counters*, apoyando la idea de que la frontera *compute-bound/bandwidth-bound* puede inferirse desde datos y no solo mediante inspección manual del modelo Roofline [10]. En conjunto, estos trabajos son importantes porque respaldan la hipótesis de que una fase de ejecución puede representarse mediante una firma microarquitectónica observable.

Sin embargo, no proponen por sí mismos un lazo de control energético que traduzca dicha clasificación en decisiones DVFS.

La cuarta línea corresponde a los mecanismos online de control dinámico durante ejecución. El antecedente más fuerte en esta categoría es DRLCAP, que propone un marco general de *GPU frequency capping* en tiempo de ejecución basado en *deep reinforcement learning*. DRLCAP monitoriza información a nivel de sistema para detectar cambios de fase, aprende una política adaptativa y reduce en promedio 22% de la energía GPU con menos de 3% de *slowdown* en arquitecturas NVIDIA, además de reportar resultados también sobre AMD [27]. Este trabajo demuestra que el control online de frecuencia GPU es factible y efectivo, pero también deja clara una diferencia conceptual frente a enfoques más ligeros: se trata de una política basada en aprendizaje por refuerzo profundo, orientada a GPU, sin formular explícitamente el problema como clasificación supervisada ligera de fases compute-bound/memory-bound seguida de una política discreta interpretable. Por tanto, aunque constituye un antecedente directo, no agota el espacio de soluciones posibles para agentes ligeros en espacio de usuario.

Finalmente, una restricción frecuentemente subestimada en la literatura es el overhead del propio mecanismo de control. Velicka et al. mostraron que la latencia de conmutación de frecuencia en GPUs no es despreciable, varía entre arquitecturas y entre pares de frecuencias, y puede llegar a ser suficientemente alta como para comprometer el beneficio de políticas excesivamente reactivas [28]. Esta observación es central, porque implica que un esquema online no debe limitarse a detectar fases correctamente: también debe amortizar el costo del cambio de frecuencia y evitar decisiones demasiado finas o frecuentes. Aun cuando la literatura reciente ha avanzado en control dinámico de frecuencia durante ejecución, la mayoría de los trabajos revisados se concentra en el dominio GPU de forma aislada. Como se discutió previamente, DRLCAP, por ejemplo, propone un marco *online* de *frequency capping* guiado por aprendizaje por refuerzo profundo, pero su problema de control está centrado en la GPU y no en una política coordinada sobre CPU y GPU dentro de un nodo heterogéneo HPC [27]. Del mismo modo, los métodos de selección óptima de frecuencia de Ali et al. y los modelos de clasificación conscientes de DVFS de Guerreiro et al. se enfocan en la selección de configuraciones para GPU, ya sea mediante modelado multiobjetivo o clasificación del comportamiento de aplicaciones GPGPU [5], [19]. Aunque existen propuestas de coordinación entre CPU, GPU y memoria, como *SparseDVFS*, estas se desarrollan en el contexto de inferencia DNN en dispositivos *edge*, con una lógica de partición por bloques y una señal de control centrada en la esparsidad de operadores, por lo que su alcance, supuestos de carga y entorno de despliegue difieren sustancialmente del problema de HPC científico general abordado aquí [29].

En conjunto, esta revisión muestra que la literatura ya dispone de componentes importantes del problema, aunque todavía de forma fragmentada: existen métodos *offline* robustos para seleccionar frecuencias óptimas, mecanismos de caracterización y clasificación de cargas, y controladores *online* avanzados para GPU. Sin embargo, sigue abierta una brecha en torno a soluciones ligeras, implementables en espacio de usuario, que integren caracterización *online* de fases de ejecución, telemetría estándar de baja intrusión y decisiones DVFS coordinadas sobre CPU y GPU en nodos heterogéneos de alto rendimiento. En ese sentido, el aporte de este trabajo no radica en afirmar que la clasificación de fases o el control *online* sean completamente inéditos, sino en proponer una integración distinta: un agente ligero, implementable sin modificaciones al *kernel*, que utilice modelos supervisados clásicos para inferir el régimen de ejecución y traduzca esa inferencia en decisiones DVFS de bajo *overhead*, con foco explícito en la optimización del Producto Energía-Retraso (EDP).

5 Metodología

La presente investigación se desarrolla bajo un enfoque cuantitativo de tipo experimental aplicado, orientado al cumplimiento del objetivo general del proyecto. El propósito metodológico consiste en determinar en qué medida un agente de gestión DVFS permite maximizar la eficiencia energética sin degradar significativamente el rendimiento de la aplicación en ejecución. Dicho impacto se evaluará a través de métricas críticas como el tiempo de ejecución y el Producto Energía-Retardo (EDP)

El desarrollo de la propuesta se estructura en cuatro fases secuenciales e interdependientes, las cuales abarcan desde la caracterización de cargas y extracción de telemetría hasta la validación empírica del sistema

5.1 Fase 1: Recolección de información y caracterización de cargas

La primera fase tiene como objetivo diseñar e implementar las técnicas de recolección de información necesarias para capturar el comportamiento dinámico del sistema. Para ello, se empleará instrumentación basada en contadores de rendimiento por hardware (HPCs), los cuales permiten registrar eventos microarquitectónicos con bajo impacto en la ejecución.

La población de estudio está constituida por el conjunto de aplicaciones ejecutables en sistemas heterogéneos CPU-GPU. No obstante, debido a la imposibilidad de abarcar la totalidad de estas, se define una muestra no probabilística de tipo intencional, compuesta por *benchmarks* y *microbenchmarks* representativos de cuatro escenarios base: CPU compute-bound, CPU memory-bound, GPU compute-bound y GPU memory-bound. Esta selección permitirá inducir regímenes de ejecución contrastantes y obtener evidencia experimental suficiente para construir un conjunto de datos útil para el entrenamiento del

clasificador. El uso de *benchmarks* representativos y de una muestra intencional es coherente con la necesidad de trabajar con cargas controladas y reproducibles dentro del alcance de un proyecto de pregrado.

Durante la ejecución de estas cargas, se implementará un sistema de recolección de datos que capturará, en intervalos periódicos, métricas relevantes del hardware. En la CPU se utilizará la interfaz *perf_event* para registrar indicadores como instrucciones retiradas, ciclos de reloj y fallo de caché de último nivel. En la GPU, se empleará la biblioteca NVML para obtener información sobre la utilización de Multiprocesadores de transmisión (SM), el uso de memoria y el consumo de potencia. Adicionalmente, se integrará mediciones energéticas mediante RALP y NVML.

Con el fin de reducir el sesgo experimental, cada benchmark se ejecutará múltiples veces bajo las mismas condiciones de afinidad, carga del sistema y configuración DVFS, descartando iteraciones iniciales de calentamiento cuando sea necesario. El resultado de esta fase será un dataset estructurado de naturaleza temporal, en el cual cada instancia representará el estado observable del sistema en una ventana de muestreo determinada, junto con la etiqueta de fase correspondiente.

5.2 Fase 2: Desarrollo del modelo de aprendizaje automático

En esta se desarrollan las técnicas de análisis orientadas a la construcción de un modelo predictivo capaz de identificar el comportamiento de la carga de trabajo en tiempo de ejecución. Inicialmente, se realizará un proceso de preprocesamiento de datos que incluye la limpieza, normalización y selección de características relevantes, con el fin de garantizar la calidad del conjunto de entrenamiento.

El problema se formula como una tarea de clasificación supervisada, donde el modelo recibe como entradas vectores de telemetría del hardware y produce como salida una etiqueta que representa la fase de ejecución de la aplicación. Esta clasificación permite inferir si el sistema se encuentra en un régimen dominado por cómputo o por memoria, información fundamental para la toma de decisiones de control energético.

Dado que el sistema debe operar en tiempo de ejecución, se prioriza el uso de un modelo de baja complejidad computacional, tales como los árboles de decisión, bosques aleatorios, entre otros, los cuales ofrecen un equilibrio adecuado entre precisión predictiva y latencia de inferencia. Durante esta fase, se entrenarán y evaluarán diferentes modelos candidatos, los cuales serán comparados utilizando métricas de clasificación y tiempo de inferencia. A partir de este análisis, se seleccionará el modelo que represente el mejor compromiso entre desempeño predictivo y costo computacional, garantizando que su ejecución no introduzca una sobrecarga significativa que

contrarreste los beneficios energéticos del sistema. El modelo seleccionado será posteriormente serializado para su integración dentro del sistema de control.

5.3 Fase 3: Implementación del agente de control DVFS

La tercera fase corresponde al desarrollo de la estrategia de implementación de la propuesta, materializada en un agente de control en espacio usuario. Este agente operará como un servicio en segundo plano encargado de monitorear constantemente el estado del hardware y aplicar decisiones de control basadas en el modelo de aprendizaje automático.

El funcionamiento del sistema se basa en un ciclo iterativo en el cual, en cada instante de ejecución, se capturan las métricas actuales del sistema, se construye el vector de entrada para el modelo y se ejecuta la inferencia correspondiente. A partir de la predicción obtenida, el agente determina la configuración de frecuencia más adecuada y la aplica mediante las interfaces disponibles del sistema operativo.

El control de frecuencia en CPU se realizará mediante herramientas de usuario compatibles con el *governor userspace*, tales como *cpupower*, para solicitar cambios de frecuencia o de P-state según las capacidades de la plataforma. En GPU, el agente utilizará *nvidia-smi* o la interfaz disponible equivalente para fijar límites de frecuencia sobre el acelerador. La política de control será discreta, es decir, no resolverá un problema continuo de optimización, sino que seleccionará un estado DVFS dentro del conjunto de configuraciones soportadas por el hardware. Esta decisión se tomará de forma proactiva, en el sentido de que estará guiada por la fase de ejecución inferida por el modelo y no únicamente por métricas reactivas instantáneas de utilización. La descripción del bucle de control y de los actuadores retoma y mejora lo ya planteado en el borrador previo, donde se explicitaba el papel de *cpupower* y *nvidia-smi* como interfaces de actuación en tiempo de ejecución.

5.4 Fase 4: Validación experimental y análisis de resultados

La fase final tiene como objetivo evaluar empíricamente el impacto del sistema propuesto sobre el consumo energético y el rendimiento, en concordancia con el cuarto objetivo específico. Para ello, se ejecutarán los benchmarks seleccionados bajo varios escenarios de referencia: (i) gobernador nativo del sistema operativo, (ii) configuración de frecuencia fija de alto rendimiento, y (iii) ejecución orquestada por el agente propuesto. Cuando la plataforma lo permita, podrán incorporarse configuraciones adicionales de referencia si ello fortalece la comparación experimental.

En cada ejecución se medirán al menos las siguientes variables: tiempo total de ejecución, energía consumida por CPU y GPU, potencia media, EDP y overhead del

agente. A partir de estas mediciones se realizará una comparación entre escenarios para determinar si el sistema propuesto mejora la eficiencia energética sin introducir una penalización severa del rendimiento. Dado que el objetivo no es solo reportar métricas, sino establecer si las diferencias observadas son estadísticamente defendibles, los resultados serán analizados mediante técnicas estadísticas apropiadas según la distribución y homogeneidad de los datos.

6 Alcance y limitaciones

6.1 Alcance

El presente proyecto se circunscribe al diseño, implementación y evaluación experimental de un agente de control DVFS en espacio de usuario para sistemas heterogéneos CPU–GPU, orientado a mejorar la eficiencia energética de aplicaciones científicas mediante modelos ligeros de aprendizaje automático. El desarrollo se realizará en un entorno de ejecución local sobre un único nodo computacional híbrido, compuesto por un procesador x86 multinúcleo con soporte para interfaces de observabilidad energética basadas en RAPL y una GPU NVIDIA accesible a través de NVML. En consecuencia, el trabajo se enfocará en la gestión intra-nodo de frecuencia y en la caracterización de la interacción entre cómputo, memoria y consumo energético dentro de ese dominio de ejecución.

Desde la perspectiva de software de sistemas, la solución propuesta operará exclusivamente en espacio de usuario, sin intervenir ni modificar directamente componentes internos del kernel del sistema operativo. El agente será implementado utilizando lenguajes y bibliotecas de propósito general apropiados para instrumentación, recolección de telemetría, inferencia y control, privilegiando alternativas ligeras y reproducibles como Python o C++ junto con interfaces estándar del ecosistema Linux. En este marco, la observabilidad del sistema se sustentará en la lectura de contadores de rendimiento por hardware e interfaces de potencia ampliamente utilizadas en la literatura, particularmente `perf_event` y RAPL para CPU, y NVML para GPU.

El núcleo predictivo del sistema se limitará al uso de modelos clásicos de aprendizaje automático supervisado de baja complejidad, tales como Árboles de Decisión, Bosques Aleatorios u otros clasificadores ligeros equivalentes cuya latencia de inferencia resulte compatible con un esquema de control en línea. El propósito de esta decisión no es maximizar complejidad algorítmica, sino preservar la viabilidad práctica del agente, minimizando la sobrecarga computacional asociada al proceso de decisión. En este sentido, el proyecto no se orienta al desarrollo de un modelo universal de optimización energética, sino a la construcción de un mecanismo experimentalmente defendible que

permita inferir fases de ejecución y traducir esa inferencia en políticas discretas de DVFS con bajo overhead.

En el plano experimental, el trabajo abarcará la ejecución controlada de benchmarks y microbenchmarks representativos de distintos regímenes de ejecución, así como de aplicaciones científicas básicas suficientemente acotadas para un proyecto de pregrado. La evaluación del sistema se realizará mediante métricas de tiempo de ejecución, energía consumida y Producto Energía–Retardo (EDP), comparando el comportamiento del agente propuesto frente a configuraciones de referencia del sistema operativo, tales como gobernadores nativos de Linux y escenarios de frecuencia fija. Por tanto, el alcance del proyecto no consiste únicamente en implementar un daemon funcional, sino en demostrar empíricamente, dentro del entorno seleccionado, si el uso de telemetría microarquitectónica y modelos ligeros de clasificación puede contribuir a una gestión energética más eficiente que las políticas reactivas tradicionales.

6.2 Limitaciones

Con el fin de preservar la viabilidad técnica, metodológica y temporal del proyecto dentro del marco de un trabajo de grado de pregrado, se establecen restricciones explícitas sobre el alcance algorítmico, arquitectónico y experimental de la propuesta. En primer lugar, se excluye de manera deliberada el desarrollo, modificación o instrumentación de componentes en kernel-space, incluyendo drivers, gobernadores del sistema operativo y mecanismos internos del planificador de frecuencia. En consecuencia, el agente propuesto actuará únicamente a través de interfaces expuestas por el sistema operativo y por las bibliotecas estándar disponibles en espacio de usuario, sin alterar la lógica interna del subsistema de administración de potencia de Linux.

Desde la perspectiva del aprendizaje automático, el proyecto no contempla el uso de arquitecturas profundas, técnicas de Aprendizaje por Refuerzo Profundo ni esquemas complejos de optimización en línea que impliquen altos costos de entrenamiento o inferencia. Esta exclusión responde a una decisión metodológica y no a una omisión accidental: dado que el objetivo del sistema es reducir consumo energético sin introducir una penalización severa en el rendimiento, la incorporación de modelos de elevada complejidad desviaría el foco del trabajo y comprometería la validez del análisis del overhead. En consecuencia, la investigación se restringe a clasificadores supervisados ligeros cuya implementación, entrenamiento e integración resulten compatibles con un entorno de control de baja latencia.

En cuanto a la infraestructura física, la propuesta se limita a plataformas con soporte para RAPL en CPU y a aceleradores NVIDIA instrumentables mediante NVML. Por tanto, no se evaluará la portabilidad del sistema hacia otros ecosistemas de hardware o software, tales como AMD ROCm, Intel oneAPI/SYCL u otras arquitecturas heterogéneas

con mecanismos distintos de telemetría y control. Esta restricción implica que cualquier generalización de resultados hacia plataformas no incluidas deberá interpretarse con cautela, dado que diferencias en el modelo de sensores, en la granularidad del control DVFS o en las interfaces de gestión pueden alterar de forma significativa tanto la observabilidad como la capacidad de actuación del agente.

Asimismo, el proyecto no contempla la ejecución en entornos distribuidos ni en clústeres multinodo como objeto directo de experimentación. Aunque el problema estudiado es pertinente para infraestructuras HPC de mayor escala, la validación del sistema se restringirá a un único nodo heterogéneo, de modo que fenómenos asociados a comunicación inter-nodo, escalabilidad distribuida, interferencia del planificador global o reparto coordinado de potencia entre múltiples nodos quedan fuera del alcance de este trabajo. En consecuencia, la contribución del proyecto debe entenderse como una aproximación intra-nodo orientada al control local de frecuencia, y no como una solución integral de gestión energética para clústeres completos.

Finalmente, el sistema de control propuesto no garantiza propiedades de temporización estricta equivalentes a las de un Sistema Operativo de Tiempo Real. Al operar en espacio de usuario sobre un sistema operativo de propósito general, el daemon estará inevitablemente sujeto a jitter, latencias de planificación, interrupciones, variaciones en el tiempo efectivo de muestreo y costos no deterministas de cambio de frecuencia. Esta limitación no invalida el proyecto, pero sí delimita con claridad el tipo de conclusión que puede sostenerse: el sistema podrá evaluarse como un mecanismo de control energético práctico y experimentalmente útil en entornos Linux convencionales, pero no como una arquitectura con garantías duras de tiempo real.

7 Cronograma

El plan de ejecución del proyecto se ha estructurado para un periodo académico de 16 semanas, organizado en cuatro fases consecutivas. Esta planificación responde a un enfoque incremental, en el cual se prioriza inicialmente la recolección y validación de datos del hardware, seguida del desarrollo del modelo de aprendizaje automático, la implementación del sistema de control y, finalmente, su validación experimental.

FASES	Actividades técnicas	MES 1 (Sem 1-4)	MES 2 (Sem 5-8)	MES 3 (Sem 9-12)	MES 4 (Sem 13-16)
Fase 1: Recolección de información y	Selección de benchmarks y microbenchmarks representativos				

caracterización de cargas	Desarrollo y ajuste de scripts de recolección (perf, RAPL y NVML)				
	Ejecución de cargas bajo distintos estados de frecuencia				
	Construcción, depuración y organización del dataset				
	Etiquetado de fases de ejecución				
	Redacción del capítulo de caracterización y metodología experimental				
Fase 2: Desarrollo del modelo de aprendizaje automático	Preprocesamiento y selección de características				
	Entrenamiento de modelos supervisados ligeros				
	Validación de desempeño predictivo				
	Medición de latencia de inferencia				
	Selección y serialización del modelo final				
	Redacción del capítulo de modelo predictivo y resultados preliminares				
Fase 3: Implementación del agente de control DVFS	Programación del daemon en espacio de usuario				
	Integración del modelo clasificador				
	Pruebas de actuación mediante cpupower y nvidia-smi				
	Medición del overhead del sistema propuesto				
	Redacción del capítulo de implementación del sistema				
Fase 4: Validación	Ejecución de escenarios de referencia				

experimental y análisis de resultados	Ejecución del sistema propuesto				
	Recolección de métricas de tiempo, energía y EDP				
	Comparación experimental y análisis estadístico				
	Redacción de resultados, discusión, conclusiones y revisión final				
	Revisión final del documento				

8 Presupuesto

La estimación presupuestal obedece a los costos de referencia institucional para el desarrollo de proyectos de grado. La valoración se presenta bajo la modalidad de aporte en especie, diferenciando la carga operativa humana (desarrollo de scripts y modelos de Machine Learning) de la infraestructura tecnológica requerida para la recolección de telemetría y validación del sistema.

8.1 Recursos Humanos

CARGO	NOMBRE	DEDICACIÓN	VALOR UNITARIO (Estimado)	TOTAL
Director	Gilberto Javier Diaz Toro	64 horas	\$41,000	\$2,624,000
Autor 1	Yeison Adrian Caceres Torres	320 horas	\$14,590.875	\$4,669,080
Autor 2	Ricardo Andrés Pérez Porras	320 horas	\$14,590.875	\$4,669,080
SUBTOTAL				\$11,962,160

Tabla 2. Presupuesto recursos humanos

8.2 Recursos Tecnológicos e Infraestructura

DESCRIPCIÓN	UNIDAD	VALOR UNITARIO	TOTAL
Uso de Supercomputadora (SC3)	5,000 horas-núcleo*	\$9,000	\$45,000,000
Computador personal	2 unidad	\$2,000,000	\$4,000,000
SUBTOTAL			\$49,000,000

Tabla 3. Presupuesto recursos tecnológicos e infraestructura

***Nota Aclaratoria:** La unidad "hora-núcleo" mide esfuerzo de procesamiento paralelo, no tiempo de calendario. Dado que cada experimento utiliza simultáneamente los 32 núcleos del nodo computacional, cada hora real consume 32 horas del presupuesto. Por tanto, las 5.000 horas-núcleo solicitadas equivalen a solo 156 horas reales (Wall-time) de ocupación del equipo ($5.000 / 32 = 156$), lo cual es viable dentro del cronograma académico.

8.3 Total General

CATEGORÍA	VALOR TOTAL
Recursos humanos	\$11,962,160
Recursos tecnológicos	\$49,000,000
TOTAL GENERAL PROYECTO	\$60,962,160

Tabla 4. Presupuesto total general

9 Bibliografía

[1] O. S. Simsek, J. -G. Piccinali and F. M. Ciorba, "Increasing Energy Efficiency of Astrophysics Simulations Through GPU Frequency Scaling", *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1826-1834, doi: 10.1109/SCW63240.2024.00229.

[2] M. T. Costa *et al.*, "Characterizing the Impact of GPU Power Management on an Exascale System", *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, New York, NY, USA, 2025, pp. 1524–1533. doi: 10.1145/3731599.3767702.

[3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu and Y. Kodama, "MCBound: An Online Framework to Characterize and Classify Memory/Compute-bound HPC Jobs", *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1-15, doi: 10.1109/SC41406.2024.00062.

- [4] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccione, "Evaluation of DVFS techniques on modern HPC processors and accelerators for energy-aware applications", *Concurr. Comput. Pract. Exper.*, vol. 29, no. 12, p. e4143, 2017. doi: 10.1002/cpe.4143.
- [5] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen, "An automated and portable method for selecting an optimal GPU frequency", *Future Generation Computer Systems*, vol. 149, pp. 71-88, Dec. 2023, doi: 10.1016/j.future.2023.07.011.
- [6] R. Gonzalez, B. M. Gordon, and M. A. Horowitz, "Supply and threshold voltage scaling for low power CMOS", *IEEE J. Solid-State Circuits*, vol. 32, no. 8, pp. 1210–1216, Aug. 1997. doi: 10.1109/4.604018.
- [7] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures", *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.
- [8] T.-Y. Liu, J. Guo, and B. Huang, "Efficient Cross-Platform Multiplexing of Hardware Performance Counters via Adaptive Grouping", *ACM Trans. Archit. Code Optim.*, vol. 21, no. 1, pp. 1–26, Mar. 2024. doi: 10.1145/3629525.
- [9] S.-K. Shekoffteh *et al.*, "Metric selection for GPU kernel classification", *ACM Trans. Archit. Code Optim.*, vol. 15, no. 4, pp. 1–27, Jan. 2019, doi: 10.1145/3295690.
- [10] L. Littman and T. Deakin, "Classifying Performance Bounds Using Machine Learning", in *Proc. SC25: Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2025.
- [11] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2020.
- [12] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.
- [13] S. McCarty, "Architecting Containers Part 1: Why Understanding User Space vs. Kernel Space Matters", *Red Hat Blog*, Jul. 29, 2015. [Online]. Available: <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>. [Accessed: Apr. 3, 2025].
- [14] V. M. Weaver, "Self-monitoring overhead of the Linux perf event performance counter interface", in *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 102–111.

- [15] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le, "RAPL: Memory power estimation and capping", in *Proc. 16th ACM/IEEE Int. Symp. Low Power Electron. Des. (ISLPED)*, Aug. 2010, pp. 189–194. doi: 10.1145/1840845.1840883.
- [16] NVIDIA, "NVIDIA Management Library (NVML)", *NVIDIA Developer Documentation*, 2026. [Online]. Available: <https://docs.nvidia.com/deploy/nvml-api/index.html> [Accessed: Apr. 3, 2026].
- [17] P. Thamm and U. Leser, "Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters", *arXiv preprint arXiv:2505.09375*, 2025.
- [18] R. Hebbar and A. Milenković, "PMU-events-driven DVFS techniques for improving energy efficiency of modern processors", *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 7, no. 1, pp. 1–31, May 2022. doi: 10.1145/3538645.
- [19] J. Guerreiro, N. Roma, P. Tomás, F. Pratas, L. S. G. Carvalho, and G. Gaydadjiev, "DVFS-aware application classification to improve GPGPUs energy efficiency", *Parallel Comput.*, vol. 83, pp. 93–117, May 2019. doi: 10.1016/j.parco.2018.02.001.
- [20] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen, "An automated and portable method for selecting an optimal GPU frequency", *Future Gener. Comput. Syst.*, vol. 149, pp. 71–88, Dec. 2023, doi: 10.1016/j.future.2023.07.011.
- [21] ScienceDirect, "Computational Kernel", *Computer Science Topics*, 2026. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/computational-kernel>. [Accessed: Apr. 3, 2026].
- [22] D. Pandey, K. Niwaria, and B. Chourasia, "Machine Learning Algorithms: A Review", *Int. J. Mach. Learn. Netw. Appl.*, vol. 6, no. 2, pp. 916–922, Jan. 2019. doi: 10.21275/ART20203995.
- [23] IBM, "What is random forest?" *IBM Think Topics*, 2026. [Online]. Available: <https://www.ibm.com/think/topics/random-forest>. [Accessed: Apr. 3, 2026].
- [24] IBM, "What is logistic regression?" *IBM Think Topics*, 2026. [Online]. Available: <https://www.ibm.com/think/topics/logistic-regression>. [Accessed: Apr. 3, 2026].
- [25] IBM, "What is XGBoost?", *IBM Think Topics*, 2026. [Online]. Available: <https://www.ibm.com/think/topics/xgboost>. [Accessed: Apr. 3, 2026].
- [26] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan, "Energy Delay Product" in *Energy-Efficient High Performance Computing*:

Measurement and Tuning, SpringerBriefs in Computer Science. London, UK: Springer, 2013, cap. 6, pp. 51–55, doi: 10.1007/978-1-4471-4492-2_9.

[27] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang, "Drlcap: Runtime GPU Frequency Capping with Deep Reinforcement Learning", *IEEE Trans. Sustain. Comput.*, vol. 9, no. 5, pp. 712–726, Sept.-Oct. 2024. doi: 10.1109/TSUSC.2024.3361596.

[28] D. Velicka, O. Vysocky, and L. Riha, "Methodology for GPU frequency switching latency measurement", in *Proc. 2025 IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, 2025, pp. 830–839. doi: 10.1109/IPDPSW66978.2025.00133.

[29] Z. Zhang, Z. Wu, J. Liu, and L. Mottola, "SparseDVFS: Sparse-Aware DVFS for Energy-Efficient Edge Inference", *arXiv preprint arXiv:2603.21908*, Mar. 23, 2026.